МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ 1

з дисципліни “Функціональне програмування”

на тему « Основи функціонального програмування у Python »

Виконав:

Студент 2 курсу

групи ФеП-23

МАГОРА Максим

Перевірив:

СВАЛЕБА Сергій Андрійович

Варіант 11: Мобільний тариф

1. Мета та завдання роботи

- Зрозуміти, що таке чиста функція і референтна прозорість.
- Навчитися виявляти та ізолювати побічні ефекти.
- Переписати імперативний фрагмент у функціональному стилі.
- Застосувати typing.Callable для параметризації обчислень.

2. Опис і назви методів (функцій), реалізованих у проєкті (згідно з методичкою)

Відповідно до архітектурного підходу методичних вказівок (поділ на чисте ядро та оболонку ефектів), для Варіанта 11 («Мобільний тариф») реалізовано такі функції та типи:

А. Чисте ядро (core.py)

- **calculate_extra_minutes_cost(record: UsageRecord) -> float**
  - *Опис:* Чиста функція розрахунку понаднормових хвилин (детермінована, без I/O та мутацій).
- **calculate_extra_sms_cost(record: UsageRecord) -> float**
  - *Опис:* Чиста функція розрахунку понаднормових SMS-повідомлень.
- **calculate_extra_gb_cost(record: UsageRecord) -> float**
  - *Опис:* Чиста функція розрахунку понаднормового інтернет-трафіку (GB).
- **compose(f: Callable[[B], C], g: Callable[[A], B]) -> Callable[[A], C]**
  - *Опис:* Функція композиції для побудови складніших правил із маленьких функцій (відповідно до додаткових вправ методички).
- **make_loyalty_bonus_policy(discount_percent: float) -> BonusFn**
  - *Опис:* Фабрика функцій (політика знижок/бонусів через Callable).
- **with_monthly_cost(record: UsageRecord, ...) -> ProcessedUsageRecord**
  - *Опис:* Аналог функції with_total з методички — повертає **новий словник** (створює новий об'єкт), не мутуючи вхідні дані.
- **make_tariff_processor(*, accept: FilterFn, bonus_policy: BonusFn) -> Callable[...]**
  - *Опис:* Аналог функції make_processor з методички — функція вищого порядку, що приймає правила фільтрації та політики обчислень через typing.Callable.

Б. Оболонка побічних ефектів (app.py)

- **render_report(result: dict[str, object]) -> None**
  - *Опис:* Функція виведення (I/O, print), яка приймає чистий результат та відображає звіт (відповідно до завдання 3 методички).
- **run() -> None**
  - *Опис:* Точка входу в програму, що ініціалізує дані та викликає процесор і звіт.

3. Структура проєкту (згідно з форматом здачі)

- core.py — чисті функції предметної області.

  app.py — оболонка побічних ефектів.

- tests/test_core.py — pytest-тести на референтну прозорість (test_referential_transparency), відсутність мутацій (test_no_mutation) та роботу політик (test_callable_policies).
- README.md — опис виконаної роботи та інструкція щодо запуску.

Висновки

Під час виконання лабораторної роботи №1 було детально досліджено та практично закріплено принципи функціонального програмування на мові Python. Зокрема, на прикладі Варіанта 11 («Мобільний тариф») було зроблено такі висновки:

1. **Забезпечення чистоти функцій та референтної прозорості:** Реалізовані функції розрахунку понаднормових витрат (calculate_extra_minutes_cost, calculate_extra_sms_cost, calculate_extra_gb_cost) є повністю детермінованими. Вони завжди повертають однаковий результат за однакових вхідних аргументів і не залежать від зовнішнього або глобального стану, що гарантує виконання принципу референтної прозорості.
2. **Ізоляція побічних ефектів (I/O та мутацій):** Усі операції з введення-виведення даних (виведення в консоль через print) було повністю винесено з обчислювального ядра в окремий модуль оболонки (app.py у функцію render_report). Завдяки використанню функції with_monthly_cost, яка замість зміни існуючих словників формує

та повертає нові об'єкти, вдалося повністю уникнути мутацій вхідних даних.

3. **Параметризація обчислень через typing.Callable:** Використання функцій вищого порядку (таких як make_tariff_processor та фабрики make_loyalty_bonus_policy) дозволило відокремити загальну схему обробки колекції даних від конкретних бізнес-правил (фільтрації та нарахування знижок). Це забезпечило високий рівень модульності, гнучкості та можливості повторного використання коду.
4. **Тестування:** За допомогою написаних модульних pytest-тестів (test_referential_transparency, test_no_mutation,test_callable_policies) було автоматично перевірено стабільність роботи чистого ядра, відсутність побічних ефектів у вигляді мутацій та коректність застосування переданих через Callable політик.